

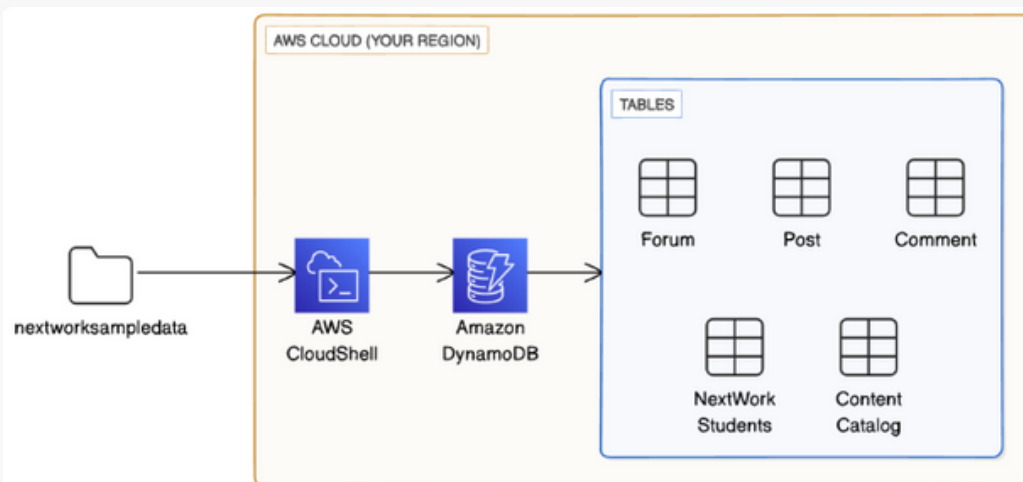


Nikko Cajayon
Community Lead



Cloud Project:

Hands-On: Best Practices for Data Loading into DynamoDB



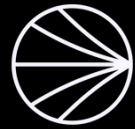


TABLE OF CONTENTS

1

INTRODUCTION

Introduction
Tools and services
learning Objectives

2

CREATING OUR DYNAMODB TABLE

Understanding Data concept
Provisioning AWS DynamoDB

3

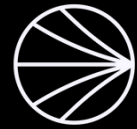
CREATE DYNAMODB TABLES WITH AWS CLOUDSHELL

Understanding No-SQL Database
Creating Database Table

4

LOAD DATA INTO YOUR TABLES

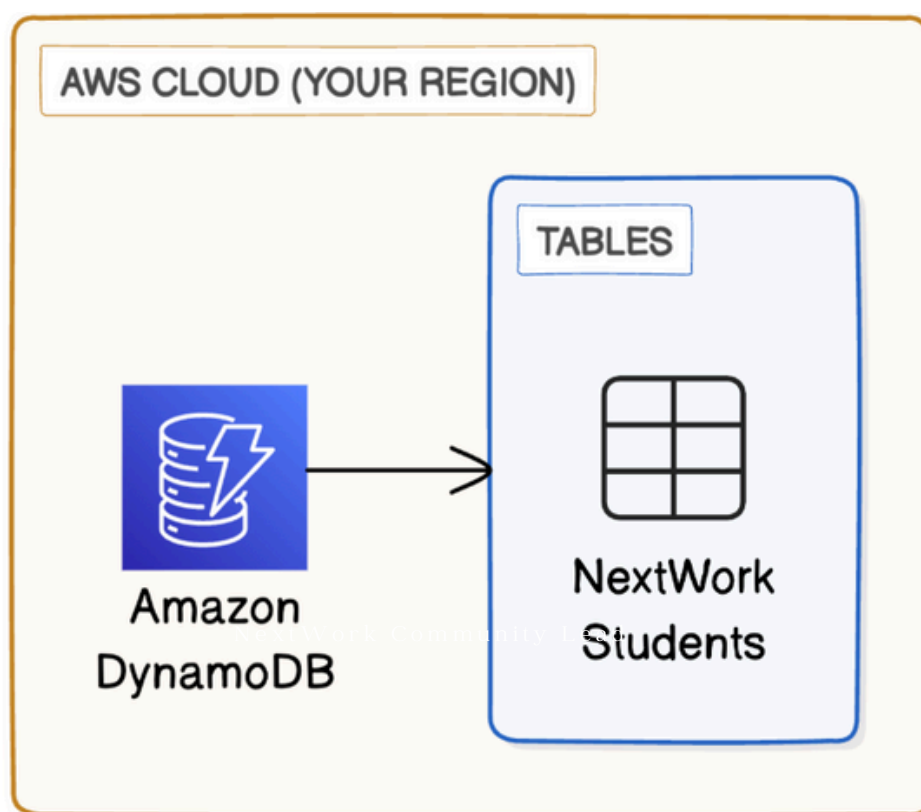
Populate Database through CLI
security best practice

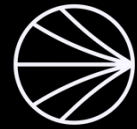


1 INTRODUCTION

You've joined the NextWork team as a Data Engineer, and your first task is to set up a NoSQL database to efficiently store unstructured and high-volume data, including user projects, video metadata, and community activity streams.

This project focuses on leveraging Amazon DynamoDB—AWS's fast and flexible NoSQL key-value database service. You will learn the fundamental skills required for a Data Engineer: creating a DynamoDB table, uploading initial datasets into it using the robust AWS Command Line Interface (CLI), and performing data manipulation tasks directly within the AWS CloudShell environment. This project marks a significant shift from the structured relational model to the flexible NoSQL paradigm.





TOOLS AND SERVICES

Amazon DynamoDB

- A fully managed NoSQL key-value and document database that delivers single-digit millisecond performance at any scale.

AWS CloudShell

- A browser-based shell that provides a Linux environment with pre-authenticated AWS credentials and the AWS CLI already installed..

AWS CLI

- AWS Command Line Interface; a unified tool to manage your AWS services from the terminal

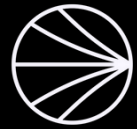
IAM (Identity and Access Management)

- Manages user roles and policies for AWS resources.
- Provides least-privilege access -
- Ensures only authorized users can manage or query the database.

LEARNING OBJECTIVES

By completing this project, we will achieve proficiency in the following:

- **NoSQL Table Design:** Creating a DynamoDB table and defining its primary key structure (Partition Key and Sort Key).
- **Data Ingestion:** Utilizing the AWS CLI to efficiently upload bulk data (JSON files) into a DynamoDB table.
- **AWS CloudShell Mastery:** Effectively using the browser-based CloudShell environment to execute management and data commands without requiring a local setup.
- **Data Manipulation:** Performing essential read and write operations (e.g., editing, querying) to manage data directly within the DynamoDB service.



2

CREATING OUR DYNAMODB TABLE

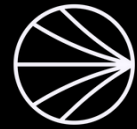
We need to build our DynamoDB tables first. This is the most important step because the way we set up each table's basic structure (like its main ID and its speed settings) directly controls how quickly and efficiently we can find the data later.

WHAT IS AWS DYNAMODB?

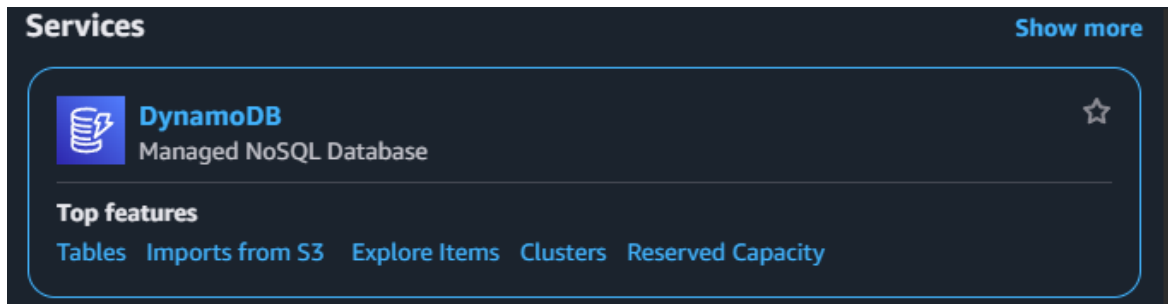
is a fully managed, serverless, key-value and document database offered by AWS. It's designed for high performance at any scale. This means it can handle a massive amount of data and traffic while maintaining fast and predictable performance.

DynamoDB is a NoSQL database, which differs from traditional relational databases. Here's a breakdown:

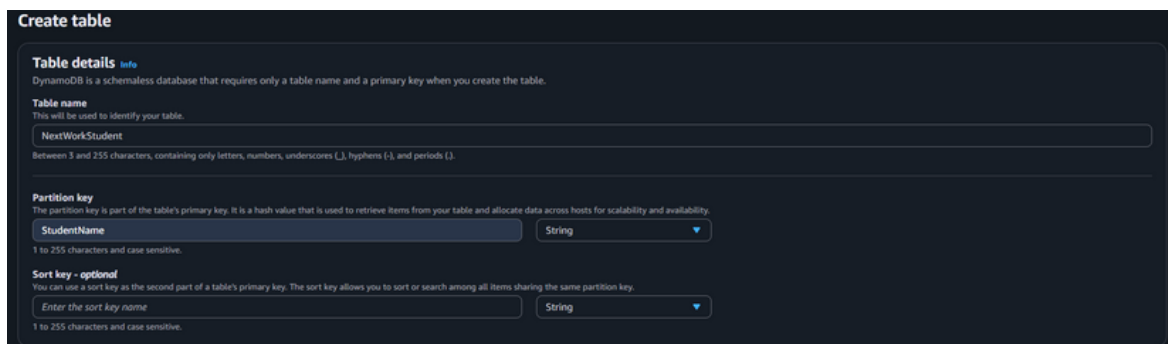
- **Key-Value and Document Database:** Data is stored in JSON-like documents, making it flexible and easy to work with complex data structures.
- **Serverless:** You don't have to manage any servers. AWS handles the infrastructure, patching, and backups.
- **Scalability:** DynamoDB can automatically scale up or down based on your application's needs, ensuring consistent performance.
- **Performance:** It's designed for low-latency reads and writes, making it suitable for applications that require real-time access to data.
- **NoSQL:** Unlike relational databases, DynamoDB doesn't use SQL for querying. It uses different methods optimized for speed and scalability.



PROVISION DYNAMO DATABASE



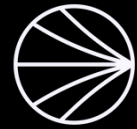
Inside of the DynamoDB console, we need to create a table that serves as the container of our data, requiring us to define the Partition Key and the Sort Key to uniquely identify each item and enable efficient querying.



WHAT IS A PARTITION KEY?

Think of a partition key as the filter that DynamoDB will use to split up and find data.

Partition key values don't have to be unique. For example if "Color" is a partition key, items can share partition key values like "Blue", "Green", "Red" and more.



CUSTOMIZE SETTINGS.

Table class [Info](#)
Select table class to optimize your table's cost based on your workload requirements and data access patterns.

Choose table class

☒ **DynamoDB Standard**
The default general-purpose table class. Recommended for the vast majority of tables that store frequently accessed data, with throughput (reads and writes) as the dominant table cost.

☐ **DynamoDB Standard-IA**
Recommended for tables that store data that is infrequently accessed, with storage as the dominant table cost.

▼ Capacity calculator [Info](#)

Average item size (KB)
1

Item read/second
1

Item write/second
1

Read consistency
Eventually consistent

Write consistency
Standard

Read capacity units
1

Write capacity units
1

Region
ap-southeast-1

Estimated cost
\$0.67 / month

Read/write capacity settings [Info](#)

Capacity mode

☒ **On-demand**
Simplify billing by paying for the actual reads and writes your application performs.

☐ **Provisioned**
Manage and optimize your costs by allocating read/write capacity in advance.

► Maximum table throughput - optional
Specify maximum on-demand read or write (or both) throughput for tables and secondary indexes to help keep your throughput usage and costs bounded. By default, maximum table throughput does not apply and on-demand throughput is only limited by the default DynamoDB table quotas.

WHY IS THERE A COST ESTIMATE?

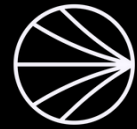
AWS cost estimates always tell you how much a resource might cost you money based on your current settings, but this estimate doesn't consider the AWS Free Tier.

WHAT IS ITEM READ/SECOND?

When we say "1 item read/second" in DynamoDB, it means your application can retrieve one item of data from your database every second.

WHAT IS ITEM WRITE/SECOND?

Just like item reads, 1 item write/second means your app can save/update one item a second.



WHAT IS AUTO SCALING?

Auto scaling can automatically adjust your database's performance (i.e. how fast it can return query results) based on real-time demand.

You'll spot auto scaling available in other services like Amazon EC2 too, where they can automatically adjust the number of instances your application needs.

AUTO SCALING SOUNDS HANDY! WHY ARE WE TURNING IT OFF?

Auto scaling can help engineers reduce costs in other situations, butttttt if it's not monitored, it also has the power to boost your table's processing power (e.g. if your table suddenly needs to update lots of data). If that ever happens, auto scaling can push your table's settings to go over Free Tier limits!

We're better off making our table stick to our current settings, which are already the lowest cost options.

Read/write capacity settings [Info](#)

Capacity mode

☐ On-demand
Simplify billing by paying for the actual reads and writes your application performs.

☒ Provisioned
Manage and optimize your costs by allocating read/write capacity in advance.

Read capacity

Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☐ On
☒ Off

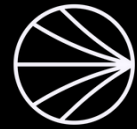
Provisioned capacity units
5

Write capacity

Auto scaling [Info](#)
Dynamically adjusts provisioned throughput capacity on your behalf in response to actual traffic patterns.

☐ On
☒ Off

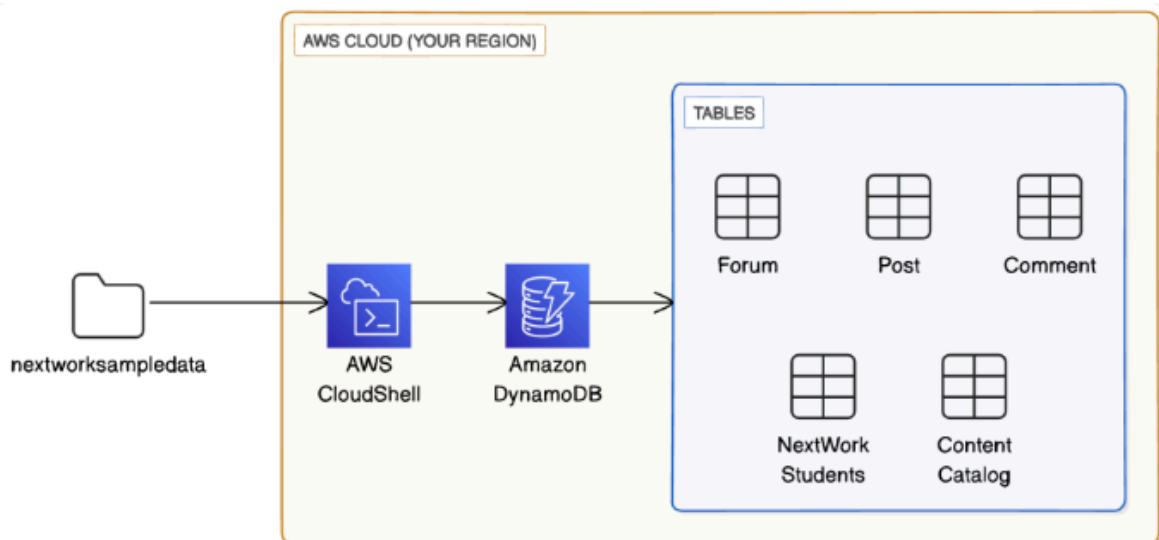
Provisioned capacity units
5



3

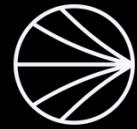
CREATE DYNAMODB TABLES WITH AWS CLOUDSHELL

This step details the process of using AWS CloudShell to execute bulk data imports, significantly improving the efficiency of loading large datasets into DynamoDB. This approach allows us to utilize the powerful AWS CLI directly from a secure, pre-configured environment.



WHAT IS CLOUDSHELL?

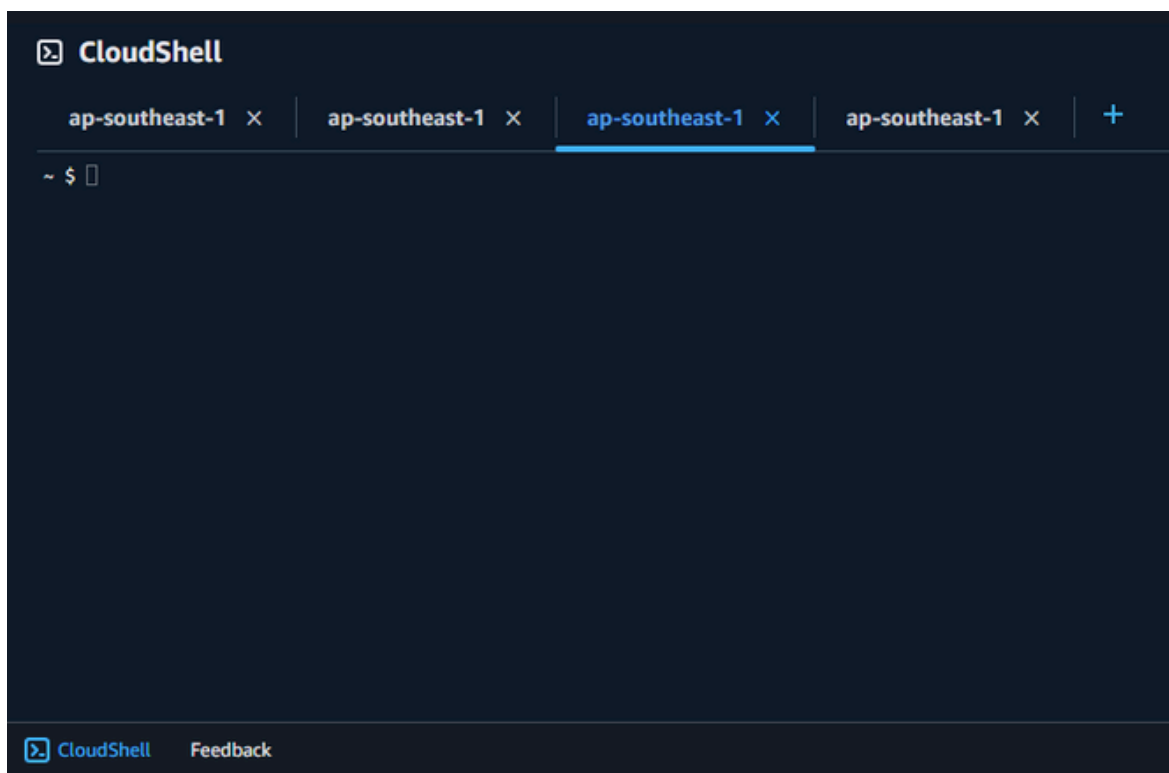
AWS CloudShell is shell in your AWS Management Console, which means it's a space for you to run code! The awesome thing about AWS CloudShell is that it already has AWS CLI pre-installed.

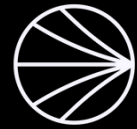


WHAT IS AWS CLI?

AWS CLI (Command Line Interface) is a software that lets you create, delete and update AWS resources with commands instead of clicking through your console.

As you advance in your cloud engineering career, you'll find that the AWS CLI often becomes your go-to tool. Engineers use the CLI to automate tasks and manage AWS resources efficiently using scripts, making it essential for managing your cloud environment in an efficient way.





RUNNING COMMANDS ON CLI

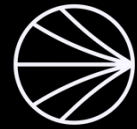
```
CloudShell
ap-northeast-1 +
> AttributeName=Subject,AttributeType=S \
> --key-schema \
> AttributeName=ForumName,KeyType=HASH \
> AttributeName=Subject,KeyType=RANGE \
> --provisioned-throughput \
> ReadCapacityUnits=1,WriteCapacityUnits=1 \
> --query "TableDescription.TableStatus"
"CREATING"
~ $ aws dynamodb create-table \
> --table-name Comment \
> --attribute-definitions \
> AttributeName=Id,AttributeType=S \
> AttributeName=CommentDateTime,AttributeType=S \
> --key-schema \
> AttributeName=Id,KeyType=HASH \
> AttributeName=CommentDateTime,KeyType=RANGE \
> --provisioned-throughput \
> ReadCapacityUnits=1,WriteCapacityUnits=1 \
> --query "TableDescription.TableStatus"
"CREATING"
~ $
```

WHAT DOES THIS COMMAND DO?

This script includes commands to create four new tables in AWS DynamoDB, each with specific attributes and settings.

The four tables created are:

1. ContentCatalog Table: This table has a numeric attribute called Id.
2. Forum Table: This table has a partition key called Name.
3. Post Table: This table has a partition key called ForumName and a sort key called Subject. We'll dive into sort keys soon!
4. Comment Table: This table has a partition key called Id and a sort key called CommentDateTime.



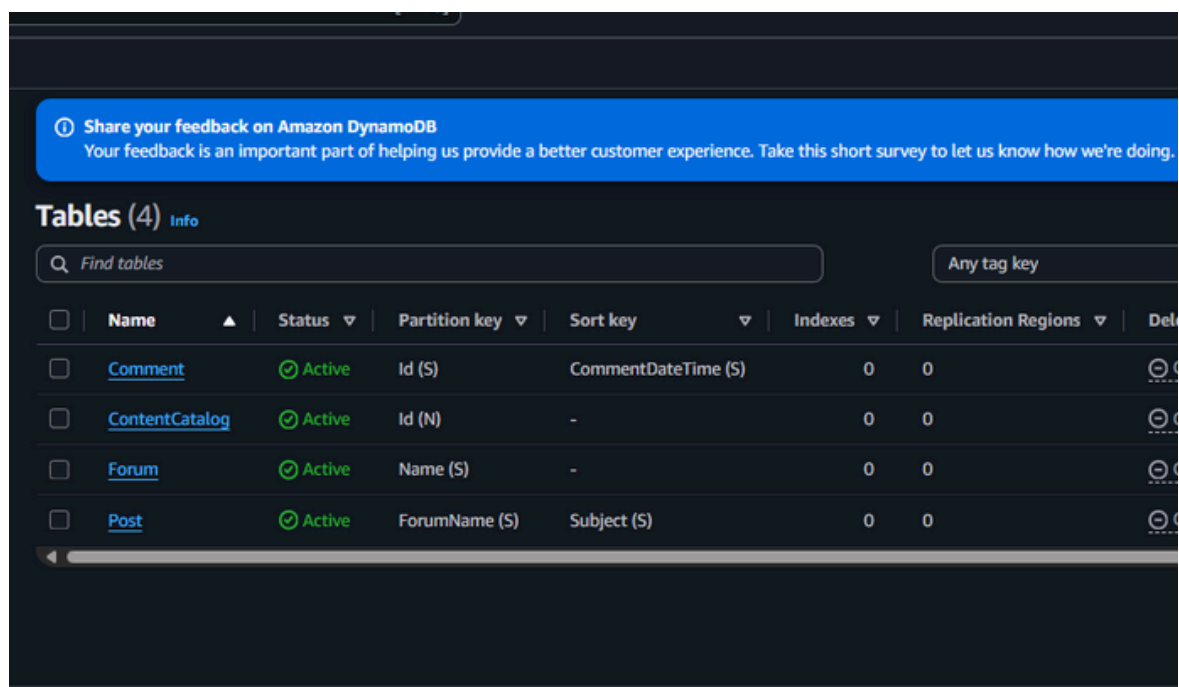
RUNNING COMMANDS ON CLI

```
aws dynamodb wait table-exists --table-name ContentCatalog
aws dynamodb wait table-exists --table-name Forum
aws dynamodb wait table-exists --table-name Post
aws dynamodb wait table-exists --table-name Comment
```

WHAT DOES THIS COMMAND DO?

When you run a wait command, you're telling your terminal to keep waiting until a condition is finally met. In our case, we're saying "keep waiting - don't finish running this command until this table has been created."

Wait commands are helpful for making sure necessary resources have been created before you move on. Otherwise, future commands that depend on your resources would automatically fail!

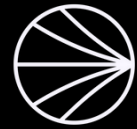


Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.

Tables (4) [Info](#)

Find tables Any tag key

☐	Name	Status	Partition key	Sort key	Indexes	Replication Regions	Del
☐	Comment	Active	Id (S)	CommentDateTime (S)	0	0	
☐	ContentCatalog	Active	Id (N)	-	0	0	
☐	Forum	Active	Name (S)	-	0	0	
☐	Post	Active	ForumName (S)	Subject (S)	0	0	



4

LOAD DATA INTO YOUR TABLES

This step details the data population of the DynamoDB tables through the optimized batch-write-item command. Validation is a key component, which includes using get-item and query operations to assess the quality and performance of data access patterns, concluding with the execution of update-item to verify write and modification functionality.

```
curl -O https://storage.googleapis.com/nextwork_course_resources/courses/aws/AWS%20Project%20People%20projects/Project%3A%20Query%20Data%20with%20DynamoDB/nextworksampladata.zip
```

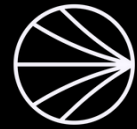
```
unzip nextworksampladata.zip
```

```
cd nextworksampladata
```

WE'RE UNZIPPING A FILE... INTO CLOUDSHELL?

That's right! CloudShell is an environment that can also handle 1GB of storage, so you can save files inside CloudShell.

And here's a fun tip: you could write and store scripts right in CloudShell to automate repetitive tasks, so you won't need to run AWS CLI commands line by line!



LOAD DATA INTO YOUR TABLES

```
MACOSX nextworksampladata nextworksampladata.zip
~ $ cd nextworksampladata
nextworksampladata $ ls
Comment.json ContentCatalog.json 'dynamodb delete-table --table-name ContentCatalog' Forum.json Post.json
nextworksampladata $ cat Forum.json
{
  "Forum": [
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "I have a question"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "2"},
          "Comments": {"N": "4"},
          "Views": {"N": "1000"}
        }
      }
    },
    {
      "PutRequest": {
        "Item": {
          "Name": {"S": "Events"},
          "Category": {"S": "NextWorkCommunity"},
          "Posts": {"N": "1"}
        }
      }
    }
  ]
}
nextworksampladata $
```

```
cd aws dynamodb batch-write-item --request-items file://ContentCatalog.json
```

```
aws dynamodb batch-write-item --request-items file://Forum.json
```

```
aws dynamodb batch-write-item --request-items file://Post.json
```

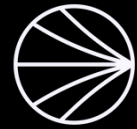
```
aws dynamodb batch-write-item --request-items file://Comment.json
```

WHAT DOES THIS COMMAND DO?

The `aws dynamodb batch-write-item` command is used to load or insert multiple items into DynamoDB tables!

`--request-items` tells DynamoDB that the items are currently stored inside a file that it'll need to retrieve from.

`file://` then tells DynamoDB that the file is stored locally in the CloudShell environment, with the name `FILENAME.json`.



LOAD DATA INTO YOUR TABLES

```
}
nextworksampldata $ aws dynamodb batch-write-item --request-items file://ContentCatalog.json
{
  "UnprocessedItems": {}
}
nextworksampldata $
nextworksampldata $ aws dynamodb batch-write-item --request-items file://Forum.json
{
  "UnprocessedItems": {}
}
nextworksampldata $
nextworksampldata $ aws dynamodb batch-write-item --request-items file://Post.json
{
  "UnprocessedItems": {}
}
nextworksampldata $
nextworksampldata $ aws dynamodb batch-write-item --request-items file://Comment.json
{
  "UnprocessedItems": {}
}
nextworksampldata $
```

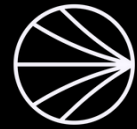
CloudShell Feedback

Table: ContentCatalog - Items returned (6)

Scan started on October 14, 2025, 11:58:52

☐	<u>Id (Number)</u>	<u>Authors</u>	<u>ContentType</u>	<u>Difficulty</u>	<u>Price</u>
☐	<u>3</u>	[{"S": "NextWork"}]	Project	Easy peasy	0
☐	<u>2</u>	[{"S": "NextWork"}]	Project	Easy peasy	0
☐	<u>203</u>		Video		0
☐	<u>202</u>		Video		0
☐	<u>201</u>		Video		0
☐	<u>1</u>	[{"S": "Natasha"}]	Project	Easy peasy	0

We have now efficiently populated our tables with the required data, achieving full data ingestion using nothing but the power of the AWS CLI available through AWS CloudShell.



PROJECT SUMMARY

This project successfully established the foundational NoSQL data store for the NextWork team, marking a shift from the traditional relational model to a highly scalable key-value service.

Key Accomplishments:

1. **DynamoDB Table Creation:** I successfully created the DynamoDB table, thoughtfully defining its Partition Key and Sort Key to ensure unique item identification and support efficient, high-speed querying for future application access patterns.
2. **Efficient Data Ingestion:** I streamlined the data loading process for large datasets by utilizing the batch-write-item command within the AWS CLI. This approach, executed entirely from AWS CloudShell, proved highly efficient, bypassing the need for complex local setup.
3. **Data Validation and Manipulation:** I validated the data integrity and access capability using core DynamoDB commands:
 - get-item and query were used to verify that data was accurately loaded and accessible via the defined keys.
 - update-item was used to confirm that necessary data modification and editing operations function correctly within the table.

By completing this project, I have demonstrated essential Data Engineering skills for working with modern cloud databases. The NextWork database is now fully provisioned and populated, and I have validated that all necessary data access and modification patterns are functional, proving that the AWS CLI within AWS CloudShell is a powerful and efficient tool for managing NoSQL infrastructure and data lifecycle.



Nikko Cajayon

NextWork Community Lead
Director for SkillBuilder
CTO of AWS Cloud Club

Here at NextWork We are
Transforming the Way World Learns

One Step Backward, Two Steps Forward